

```

import json
import time
import warnings
import numpy as np
import pandas as pd
import joblib
from joblib import Parallel, delayed

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, precision_score, recall_score, confusion_matrix

import xgboost as xgb
import lightgbm as lgb
from catboost import CatBoostClassifier
import shap

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
N_FOLDS = 5
FP_COST = 5
FN_COST = 1
np.random.seed(RANDOM_STATE)
print("✓ Setup complete")

```

✓ Setup complete

```

# For Colab: uncomment and run
# from google.colab import drive
# drive.mount('/content/drive')
# DATA_DIR = '/content/drive/My Drive/path/to/data/'

# For local: set this to your CSV folder
DATA_DIR = '/content/drive/My Drive/Datasets/'

app_train = pd.read_csv(f'{DATA_DIR}HC_application_train.csv')
assert app_train is not None, "application_train.csv required"
assert app_train['SK_ID_CURR'].duplicated().sum() == 0, "Duplicate SK_ID_CURR"

print(f"✓ Loaded: {app_train.shape}")
print(f" TARGET rate: {app_train['TARGET'].mean():.4f}")

```

✓ Loaded: (307511, 122)
TARGET rate: 0.0807

```

def engineer_application_features(df):
    df = df.copy()
    eps = 1e-6

    df['AGE_YEARS'] = df['DAYS_BIRTH'] / 365
    df['YEARS_EMPLOYED'] = df['DAYS_EMPLOYED'] / 365
    df['CREDIT_INCOME_RATIO'] = df['AMT_CREDIT'] / (df['AMT_INCOME_TOTAL'] + eps)
    df['ANNUITY_INCOME_RATIO'] = df['AMT_ANNUITY'] / (df['AMT_INCOME_TOTAL'] + eps)
    df['GOODS_CREDIT_RATIO'] = df['AMT_GOODS_PRICE'] / (df['AMT_CREDIT'] + eps)
    df['PAYMENT_RATE'] = df['AMT_ANNUITY'] / (df['AMT_CREDIT'] + eps)
    df['CREDIT_TERM_YEARS'] = df['AMT_CREDIT'] / (df['AMT_ANNUITY'] + eps)
    df['EMPLOYED_BIRTH_RATIO'] = df['DAYS_EMPLOYED'] / (df['DAYS_BIRTH'] + eps)
    df['DAYS_EMPLOYED_PERCENT'] = df['YEARS_EMPLOYED'] / (df['AGE_YEARS'] + eps)
    df['INCOME_PER_FAMILY'] = df['AMT_INCOME_TOTAL'] / df['CNT_FAM_MEMBERS'].replace(0, 1)
    df['INCOME_PER_CHILD'] = df['AMT_INCOME_TOTAL'] / (1 + df['CNT_CHILDREN'])
    df['CHILDREN_RATIO'] = df['CNT_CHILDREN'] / df['CNT_FAM_MEMBERS'].replace(0, 1)
    if 'OWN_CAR_AGE' in df.columns:
        df['CAR_AGE_RATIO'] = df['OWN_CAR_AGE'] / (df['AGE_YEARS'] + eps)

    ext_cols = [c for c in ['EXT_SOURCE_1', 'EXT_SOURCE_2', 'EXT_SOURCE_3'] if c in df.columns]
    if ext_cols:
        df['EXT_SOURCE_MEAN'] = df[ext_cols].mean(axis=1)
        df['EXT_SOURCE_STD'] = df[ext_cols].std(axis=1)
        df['EXT_SOURCE_MIN'] = df[ext_cols].min(axis=1)
        df['EXT_SOURCE_MAX'] = df[ext_cols].max(axis=1)
        df['EXT_SOURCE_NANCOUNT'] = df[ext_cols].isnull().sum(axis=1)

    doc_cols = [c for c in df.columns if c.startswith('FLAG_DOCUMENT_')]

```

```

if doc_cols:
    df['DOCUMENT_COUNT'] = df[doc_cols].sum(axis=1)

contact_cols = [c for c in ['FLAG_MOBIL', 'FLAG_EMP_PHONE', 'FLAG_WORK_PHONE',
                            'FLAG_CONT_MOBILE', 'FLAG_PHONE', 'FLAG_EMAIL'] if c in df.columns]

if contact_cols:
    df['CONTACT_FLAG_SUM'] = df[contact_cols].sum(axis=1)

return df

def impute_and_prepare(df, fit_stats=None):
    df = df.copy()
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
    categorical_cols = [c for c in categorical_cols if c not in ('SK_ID_CURR',)]

    if fit_stats is None:
        fit_stats = {'numeric_median': {}, 'categorical_mode': {}}
        for col in numeric_cols:
            if df[col].isnull().mean() > 0.05:
                df[f'{col}_ISNULL'] = df[col].isnull().astype('int8')
                fit_stats['numeric_median'][col] = df[col].median()
            for col in categorical_cols:
                fit_stats['categorical_mode'][col] = df[col].mode().iloc[0] if not df[col].mode().empty else 'Unknown'
    else:
        for col in numeric_cols:
            if col in fit_stats['numeric_median'] and df[col].isnull().mean() > 0.05:
                if f'{col}_ISNULL' not in df.columns:
                    df[f'{col}_ISNULL'] = df[col].isnull().astype('int8')

        for col in numeric_cols:
            median_val = fit_stats['numeric_median'].get(col, df[col].median())
            df[col] = df[col].fillna(median_val)
        for col in categorical_cols:
            mode_val = fit_stats['categorical_mode'].get(col, 'Unknown')
            df[col] = df[col].fillna(mode_val).astype(str)

    return df, fit_stats

def one_hot_low_card(df, exclude_cols=None, max_card=10):
    if exclude_cols is None:
        exclude_cols = []
    cat_cols = df.select_dtypes(include=['object']).columns.tolist()
    low_card = [c for c in cat_cols if c not in exclude_cols and df[c].nunique() <= max_card]
    df = pd.get_dummies(df, columns=low_card, dummy_na=False)
    return df

app_fe = engineer_application_features(app_clean)
full_df = app_fe.copy()
full_df, impute_stats = impute_and_prepare(full_df)

HIGH_CARD_COLS = ['ORGANIZATION_TYPE', 'OCCUPATION_TYPE']
full_df = one_hot_low_card(full_df, exclude_cols=HIGH_CARD_COLS)

EXCLUDE = ['SK_ID_CURR', 'TARGET']
X_full = full_df[[c for c in full_df.columns if c not in EXCLUDE]]
y_full = full_df['TARGET']

print(f"✓ Features engineered: {X_full.shape}")

```

✓ Features engineered: (307511, 211)

```

def engineer_application_features(df):
    df = df.copy()
    eps = 1e-6

    df['AGE_YEARS'] = df['DAYS_BIRTH'] / 365
    df['YEARS_EMPLOYED'] = df['DAYS_EMPLOYED'] / 365
    df['CREDIT_INCOME_RATIO'] = df['AMT_CREDIT'] / (df['AMT_INCOME_TOTAL'] + eps)
    df['ANNUITY_INCOME_RATIO'] = df['AMT_ANNUITY'] / (df['AMT_INCOME_TOTAL'] + eps)
    df['GOODS_CREDIT_RATIO'] = df['AMT_GOODS_PRICE'] / (df['AMT_CREDIT'] + eps)
    df['PAYMENT_RATE'] = df['AMT_ANNUITY'] / (df['AMT_CREDIT'] + eps)
    df['CREDIT_TERM_YEARS'] = df['AMT_CREDIT'] / (df['AMT_ANNUITY'] + eps)

```

```

df['EMPLOYED_BIRTH_RATIO'] = df['DAYS_EMPLOYED'] / (df['DAYS_BIRTH'] + eps)
df['DAYS_EMPLOYED_PERCENT'] = df['YEARS_EMPLOYED'] / (df['AGE_YEARS'] + eps)
df['INCOME_PER_FAMILY'] = df['AMT_INCOME_TOTAL'] / df['CNT_FAM_MEMBERS'].replace(0, 1)
df['INCOME_PER_CHILD'] = df['AMT_INCOME_TOTAL'] / (1 + df['CNT_CHILDREN'])
df['CHILDREN_RATIO'] = df['CNT_CHILDREN'] / df['CNT_FAM_MEMBERS'].replace(0, 1)
if 'OWN_CAR_AGE' in df.columns:
    df['CAR_AGE_RATIO'] = df['OWN_CAR_AGE'] / (df['AGE_YEARS'] + eps)

ext_cols = [c for c in ['EXT_SOURCE_1', 'EXT_SOURCE_2', 'EXT_SOURCE_3'] if c in df.columns]
if ext_cols:
    df['EXT_SOURCE_MEAN'] = df[ext_cols].mean(axis=1)
    df['EXT_SOURCE_STD'] = df[ext_cols].std(axis=1)
    df['EXT_SOURCE_MIN'] = df[ext_cols].min(axis=1)
    df['EXT_SOURCE_MAX'] = df[ext_cols].max(axis=1)
    df['EXT_SOURCE_NANCOUNT'] = df[ext_cols].isnull().sum(axis=1)

doc_cols = [c for c in df.columns if c.startswith('FLAG_DOCUMENT_')]
if doc_cols:
    df['DOCUMENT_COUNT'] = df[doc_cols].sum(axis=1)

contact_cols = [c for c in ['FLAG_MOBIL', 'FLAG_EMP_PHONE', 'FLAG_WORK_PHONE',
                             'FLAG_CONT_MOBILE', 'FLAG_PHONE', 'FLAG_EMAIL'] if c in df.columns]
if contact_cols:
    df['CONTACT_FLAG_SUM'] = df[contact_cols].sum(axis=1)

return df

def impute_and_prepare(df, fit_stats=None):
    df = df.copy()
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
    categorical_cols = [c for c in categorical_cols if c not in ('SK_ID_CURR',)]

    if fit_stats is None:
        fit_stats = {'numeric_median': {}, 'categorical_mode': {}}
        for col in numeric_cols:
            if df[col].isnull().mean() > 0.05:
                df[f'{col}_ISNULL'] = df[col].isnull().astype('int8')
                fit_stats['numeric_median'][col] = df[col].median()
        for col in categorical_cols:
            fit_stats['categorical_mode'][col] = df[col].mode().iloc[0] if not df[col].mode().empty else 'Unknown'
    else:
        for col in numeric_cols:
            if col in fit_stats['numeric_median'] and df[col].isnull().mean() > 0.05:
                if f'{col}_ISNULL' not in df.columns:
                    df[f'{col}_ISNULL'] = df[col].isnull().astype('int8')

        for col in numeric_cols:
            median_val = fit_stats['numeric_median'].get(col, df[col].median())
            df[col] = df[col].fillna(median_val)
        for col in categorical_cols:
            mode_val = fit_stats['categorical_mode'].get(col, 'Unknown')
            df[col] = df[col].fillna(mode_val).astype(str)

    return df, fit_stats

def one_hot_low_card(df, exclude_cols=None, max_card=10):
    if exclude_cols is None:
        exclude_cols = []
    cat_cols = df.select_dtypes(include=['object']).columns.tolist()
    low_card = [c for c in cat_cols if c not in exclude_cols and df[c].nunique() <= max_card]
    df = pd.get_dummies(df, columns=low_card, dummy_na=False)
    return df

app_fe = engineer_application_features(app_clean)
full_df = app_fe.copy()
full_df, impute_stats = impute_and_prepare(full_df)

HIGH_CARD_COLS = ['ORGANIZATION_TYPE', 'OCCUPATION_TYPE']
full_df = one_hot_low_card(full_df, exclude_cols=HIGH_CARD_COLS)

EXCLUDE = ['SK_ID_CURR', 'TARGET']
X_full = full_df[[c for c in full_df.columns if c not in EXCLUDE]]
y_full = full_df['TARGET']

```

```
print(f"✓ Features engineered: {X_full.shape}")
```

```
✓ Features engineered: (307511, 211)
```

Cell 4: Train Arena – Full Code with LightGBM Fix

```
TARGET_ENC_SMOOTHING = 20

def sanitize_feature_names(df):
    """Replace problematic characters in column names for LightGBM"""
    new_columns = []
    for col in df.columns:
        new_col = str(col)
        new_col = new_col.replace('[', '_').replace(']', '_')
        new_col = new_col.replace('<', '_').replace('=', '_')
        new_col = new_col.replace(' ', '_').replace(',', '_')
        new_col = new_col.replace(':', '_').replace('; ', '_')
        new_col = new_col.replace('(', '_').replace(')', '_')
        new_columns.append(new_col)
    df.columns = new_columns
    return df

def kfold_target_encode(train_df, val_df, col, target_col='TARGET', smoothing=TARGET_ENC_SMOOTHING):
    global_mean = train_df[target_col].mean()
    stats = train_df.groupby(col)[target_col].agg(['mean', 'count'])
    smoothed = (stats['count'] * stats['mean'] + smoothing * global_mean) / (stats['count'] + smoothing)
    train_enc = train_df[col].map(smoothed).fillna(global_mean)
    val_enc = val_df[col].map(smoothed).fillna(global_mean)
    return train_enc.astype('float32'), val_enc.astype('float32'), smoothed, global_mean

skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_STATE)

cat_present = [c for c in HIGH_CARD_COLS if c in X_full.columns]
numeric_and_ohe_cols = [c for c in X_full.columns if c not in cat_present]

oof_preds = {'xgb': np.zeros(len(X_full)), 'lgb': np.zeros(len(X_full)), 'cat': np.zeros(len(X_full))}
fold_metrics = {'xgb': [], 'lgb': [], 'cat': []}
models_per_fold = {'xgb': [], 'lgb': [], 'cat': []}
train_preds_cached = {}

pos = y_full.sum()
neg = len(y_full) - pos
raw_ratio = neg / pos

for fold, (train_idx, val_idx) in enumerate(skf.split(X_full, y_full)):
    print(f"\n=== Fold {fold+1}/{N_FOLDS} ===")
    X_train_raw, X_val_raw = full_df.iloc[train_idx], full_df.iloc[val_idx]
    y_train, y_val = y_full.iloc[train_idx], y_full.iloc[val_idx]

    # Encode high-cardinality categoricals
    X_train_enc, X_val_enc = X_train_raw[numeric_and_ohe_cols].copy(), X_val_raw[numeric_and_ohe_cols].copy()
    for col in cat_present:
        tr_enc, va_enc, _, _ = kfold_target_encode(X_train_raw, X_val_raw, col, target_col='TARGET')
        X_train_enc[f'{col}_TE'] = tr_enc.values
        X_val_enc[f'{col}_TE'] = va_enc.values

    # Sanitize column names for LightGBM
    X_train_enc = sanitize_feature_names(X_train_enc)
    X_val_enc = sanitize_feature_names(X_val_enc)

    # ===== XGBoost =====
    t0 = time.time()
    xgb_model = xgb.XGBClassifier(n_estimators=3000, learning_rate=0.02, max_depth=6, subsample=0.85,
                                  colsample_bytree=0.75, min_child_weight=50, reg_alpha=0.04, reg_lambda=0.075,
                                  scale_pos_weight=0.5 * raw_ratio, tree_method='hist', eval_metric='auc',
                                  early_stopping_rounds=100, n_jobs=-1, random_state=RANDOM_STATE)
    xgb_model.fit(X_train_enc, y_train, eval_set=(X_val_enc, y_val), verbose=False)
    xgb_prob = xgb_model.predict_proba(X_val_enc)[:, 1]
    oof_preds['xgb'][val_idx] = xgb_prob
    models_per_fold['xgb'].append(xgb_model)
    tn, fp, fn, tp = confusion_matrix(y_val, (xgb_prob >= 0.5).astype(int)).ravel()
    fpr = fp / (fp + tn)
    fold_metrics['xgb'].append({'auc': roc_auc_score(y_val, xgb_prob), 'fpr': fpr, 'fnr': fn/(fn+tp),
                                'precision': precision_score(y_val, (xgb_prob >= 0.5).astype(int)),
                                'recall': recall_score(y_val, (xgb_prob >= 0.5).astype(int)), 'time': time.time()-t0})
```

```

# Cache train preds for Cell 5
train_preds_cached[fold] = {}
X_train_enc_for_cache = X_train_raw[numeric_and_ohe_cols].copy()
for col in cat_present:
    tr_enc, _, _, _ = kfold_target_encode(X_train_raw, X_train_raw, col, target_col='TARGET')
    X_train_enc_for_cache[f'{col}_TE'] = tr_enc.values
X_train_enc_for_cache = sanitize_feature_names(X_train_enc_for_cache)
train_preds_cached[fold]['xgb'] = xgb_model.predict_proba(X_train_enc_for_cache)[: , 1]

# ===== LightGBM =====
t0 = time.time()
lgb_model = lgb.LGBMClassifier(n_estimators=3000, learning_rate=0.02, num_leaves=34, max_depth=6,
    feature_fraction=0.9, bagging_fraction=0.8, bagging_freq=5, min_child_samples=70,
    reg_alpha=0.04, reg_lambda=0.075, scale_pos_weight=0.5 * raw_ratio, n_jobs=-1,
    random_state=RANDOM_STATE, verbosity=-1)
lgb_model.fit(X_train_enc, y_train, eval_set=(X_val_enc, y_val),
    callbacks=[lgb.early_stopping(100, verbose=False)])
lgb_prob = lgb_model.predict_proba(X_val_enc)[: , 1]
oof_preds['lgb'][val_idx] = lgb_prob
models_per_fold['lgb'].append(lgb_model)
tn, fp, fn, tp = confusion_matrix(y_val, (lgb_prob >= 0.5).astype(int)).ravel()
fpr = fp / (fp + tn)
fold_metrics['lgb'].append({'auc': roc_auc_score(y_val, lgb_prob), 'fpr': fpr, 'fnr': fn/(fn+tp),
    'precision': precision_score(y_val, (lgb_prob >= 0.5).astype(int)),
    'recall': recall_score(y_val, (lgb_prob >= 0.5).astype(int)), 'time': time.time()-t0})
train_preds_cached[fold]['lgb'] = lgb_model.predict_proba(X_train_enc_for_cache)[: , 1]

# ===== CatBoost =====
t0 = time.time()
X_train_cat = X_train_raw[numeric_and_ohe_cols + cat_present].copy()
X_val_cat = X_val_raw[numeric_and_ohe_cols + cat_present].copy()
for c in cat_present:
    X_train_cat[c] = X_train_cat[c].astype(str)
    X_val_cat[c] = X_val_cat[c].astype(str)
cat_model = CatBoostClassifier(iterations=3000, learning_rate=0.03, depth=6, l2_leaf_reg=6,
    auto_class_weights='Balanced', cat_features=cat_present, random_state=RANDOM_STATE,
    verbose=False, early_stopping_rounds=100, thread_count=-1)
cat_model.fit(X_train_cat, y_train, eval_set=(X_val_cat, y_val))
cat_prob = cat_model.predict_proba(X_val_cat)[: , 1]
oof_preds['cat'][val_idx] = cat_prob
models_per_fold['cat'].append(cat_model)
tn, fp, fn, tp = confusion_matrix(y_val, (cat_prob >= 0.5).astype(int)).ravel()
fpr = fp / (fp + tn)
fold_metrics['cat'].append({'auc': roc_auc_score(y_val, cat_prob), 'fpr': fpr, 'fnr': fn/(fn+tp),
    'precision': precision_score(y_val, (cat_prob >= 0.5).astype(int)),
    'recall': recall_score(y_val, (cat_prob >= 0.5).astype(int)), 'time': time.time()-t0})
X_train_cat_cache = X_train_raw[numeric_and_ohe_cols + cat_present].copy()
for c in cat_present:
    X_train_cat_cache[c] = X_train_cat_cache[c].astype(str)
train_preds_cached[fold]['cat'] = cat_model.predict_proba(X_train_cat_cache)[: , 1]

# Print fold results
print(f" xgb: AUC={fold_metrics['xgb'][-1]['auc']:.4f} FPR={fold_metrics['xgb'][-1]['fpr']:.3f}")
print(f" lgb: AUC={fold_metrics['lgb'][-1]['auc']:.4f} FPR={fold_metrics['lgb'][-1]['fpr']:.3f}")
print(f" cat: AUC={fold_metrics['cat'][-1]['auc']:.4f} FPR={fold_metrics['cat'][-1]['fpr']:.3f}")

print("✓ Training complete, predictions cached for Cell 5")

```

```

=== Fold 1/5 ===
xgb: AUC=0.7671 FPR=0.104
lgb: AUC=0.7357 FPR=0.000
cat: AUC=0.7649 FPR=0.271

=== Fold 2/5 ===
xgb: AUC=0.7747 FPR=0.107
lgb: AUC=0.7404 FPR=0.000
cat: AUC=0.7733 FPR=0.275

=== Fold 3/5 ===
xgb: AUC=0.7682 FPR=0.110
lgb: AUC=0.7340 FPR=0.000
cat: AUC=0.7649 FPR=0.279

=== Fold 4/5 ===

```

```

def summarize_arena(fold_metrics):
    rows = []
    for name, folds in fold_metrics.items():
        df_f = pd.DataFrame(folds)
        rows.append({'model': name, 'auc_mean': df_f['auc'].mean(), 'auc_std': df_f['auc'].std(),
                    'fpr_mean': df_f['fpr'].mean(), 'fnr_mean': df_f['fnr'].mean(),
                    'precision_mean': df_f['precision'].mean(), 'recall_mean': df_f['recall'].mean(),
                    'time_mean': df_f['time'].mean()})
    return pd.DataFrame(rows).sort_values('auc_mean', ascending=False)

arena_summary = summarize_arena(fold_metrics)
print("=== ARENA SUMMARY ===")
print(arena_summary.to_string())

WINNER = arena_summary.iloc[0]['model']
print(f"\n✓ Winner: {WINNER.upper()} (AUC={arena_summary.iloc[0]['auc_mean']:.4f})")

def fast_threshold_sweep(y_true, y_prob, fp_cost=FP_COST, fn_cost=FN_COST, early_stop_patience=5):
    y_true = np.array(y_true, dtype=np.int32)
    y_prob = np.array(y_prob, dtype=np.float32)
    sorted_idx = np.argsort(-y_prob)
    y_sorted = y_true[sorted_idx]
    tp_cumsum = np.cumsum(y_sorted)
    fp_cumsum = np.cumsum(1 - y_sorted)
    n_neg = (y_true == 0).sum()
    n_pos = (y_true == 1).sum()
    unique_probs = np.unique(y_prob)
    unique_probs = unique_probs[(unique_probs > 0.0) & (unique_probs < 1.0)]
    unique_probs = np.sort(unique_probs)[::-1]
    best_thresh, best_cost, no_improve_count = 0.5, np.inf, 0
    results = []
    for i, thresh in enumerate(unique_probs):
        idx = np.searchsorted(-y_prob[sorted_idx], -thresh, side='left')
        tp = tp_cumsum[idx] if idx > 0 else 0
        fp = fp_cumsum[idx] if idx > 0 else 0
        fn = n_pos - tp
        tn = n_neg - fp
        cost = fp_cost * fp + fn_cost * fn
        results.append({'threshold': thresh, 'cost': cost, 'fp': fp, 'fn': fn, 'tp': tp, 'tn': tn})
        if cost < best_cost:
            best_cost, best_thresh = cost, thresh
            no_improve_count = 0
        else:
            no_improve_count += 1
            if no_improve_count >= early_stop_patience:
                print(f" [Early Stop] No improvement for {early_stop_patience} steps")
                break
    return best_thresh, pd.DataFrame(results)

best_threshold, threshold_sweep = fast_threshold_sweep(y_full.values, oof_preds[WINNER])
print(f"\n✓ Cost-optimal threshold (FP={FP_COST}x FN): {best_threshold:.4f}")

final_pred = (oof_preds[WINNER] >= best_threshold).astype(int)
tn, fp, fn, tp = confusion_matrix(y_full, final_pred).ravel()
fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
precision = tp / (tp + fp) if (tp + fp) > 0 else 0

print(f"\n=== OOF METRICS @ TUNED THRESHOLD ===")
print(f"Confusion: TN={tn} FP={fp} FN={fn} TP={tp}")
print(f"FPR={fpr:.4f} Recall={recall:.4f} Precision={precision:.4f}")
print(f"Cost: {FP_COST * fp + FN_COST * fn}")

def compute_fold_auc_gap(fold, train_idx, val_idx, y_full, train_preds_cached, fold_metrics, WINNER):
    train_prob = train_preds_cached[fold][WINNER]
    y_train = y_full.iloc[train_idx]
    train_auc = roc_auc_score(y_train, train_prob)
    val_auc = fold_metrics[WINNER][fold]['auc']
    gap = train_auc - val_auc
    return fold, train_auc, val_auc, gap

print(f"\n=== ANTI-OVERFIT CHECK (Cached) ===")
fold_gaps = Parallel(n_jobs=-1)(
    delayed(compute_fold_auc_gap)(fold, train_idx, val_idx, y_full, train_preds_cached, fold_metrics, WINNER)
    for fold, (train_idx, val_idx) in enumerate(skf.split(X_full, y_full))
)

```

```

max_gap = 0
for fold, train_auc, val_auc, gap in fold_gaps:
    flag = "✓ OK" if gap <= 0.03 else "△ FAIL"
    print(f"Fold {fold+1}: train={train_auc:.4f} val={val_auc:.4f} gap={gap:.4f} [{flag}]")
    max_gap = max(max_gap, gap)

assert max_gap <= 0.05, f"Overfit {max_gap:.4f} exceeds 0.05"
print(f"✓ No overfit (max gap: {max_gap:.4f})")

```

```

print("Computing SHAP values (sample 2000 rows)...")
sample_idx = np.random.RandomState(RANDOM_STATE).choice(len(X_full), size=min(2000, len(X_full)), replace=False)

if WINNER == 'cat':
    explain_frame = full_df.iloc[sample_idx][numeric_and_ohe_cols + cat_present]
    explainer = shap.TreeExplainer(models_per_fold['cat'][0])
else:
    explain_frame = full_df.iloc[sample_idx][numeric_and_ohe_cols]
    explainer = shap.TreeExplainer(models_per_fold[WINNER][0])

shap_values = explainer.shap_values(explain_frame)
try:
    shap.summary_plot(shap_values, explain_frame, max_display=15, show=False)
    print("✓ SHAP plot saved")
except Exception as e:
    print(f"SHAP plot skipped: {e}")

```

```

def predict_score(input_dict, artifact_path='finguard_model.pkl'):
    art = joblib.load(artifact_path)
    row = pd.DataFrame([input_dict])
    print("Training final model on 100% data...")
    best_iters = []
    for m in models_per_fold[WINNER]:
        if WINNER == 'xgb':
            best_iters.append(m.best_iteration)
        elif WINNER == 'lgb':
            best_iters.append(m.best_iteration_)
        else:
            best_iters.append(m.get_best_iteration())
    final_n_estimators = int(np.median(best_iters))
    print(f"Using {final_n_estimators} estimators")

    full_target_enc_maps = {}
    for col in cat_present:
        global_mean = y_full.mean()
        stats = full_df.groupby(col)['TARGET'].agg(['mean', 'count'])
        smoothed = (stats['count'] * stats['mean'] + TARGET_ENC_SMOOTHING * global_mean) / (stats['count'] + TARGET_ENC_SMOOTHING)
        full_target_enc_maps[col] = {'map': smoothed.to_dict(), 'global_mean': float(global_mean)}
        full_df[f'{col}_TE'] = full_df[col].map(smoothed).fillna(global_mean).astype('float32')

    final_feature_cols = numeric_and_ohe_cols + [f'{c}_TE' for c in cat_present]
    X_final = full_df[final_feature_cols]

    if WINNER == 'xgb':
        final_model = xgb.XGBClassifier(n_estimators=final_n_estimators, learning_rate=0.02, max_depth=6,
            subsample=0.85, colsample_bytree=0.75, min_child_weight=50, reg_alpha=0.04, reg_lambda=0.075,
            scale_pos_weight=0.5 * raw_ratio, tree_method='hist', n_jobs=-1, random_state=RANDOM_STATE)
        final_model.fit(X_final, y_full)
    elif WINNER == 'lgb':
        final_model = lgb.LGBMClassifier(n_estimators=final_n_estimators, learning_rate=0.02, num_leaves=34,
            max_depth=6, feature_fraction=0.9, bagging_fraction=0.8, bagging_freq=5, min_child_samples=70,
            reg_alpha=0.04, reg_lambda=0.075, scale_pos_weight=0.5 * raw_ratio, n_jobs=-1,
            random_state=RANDOM_STATE, verbosity=-1)
        final_model.fit(X_final, y_full)
    else:
        X_final = full_df[numeric_and_ohe_cols + cat_present]
        for c in cat_present:
            X_final[c] = X_final[c].astype(str)
        final_model = CatBoostClassifier(iterations=final_n_estimators, learning_rate=0.03, depth=6,
            l2_leaf_reg=6, auto_class_weights='Balanced', cat_features=cat_present,
            random_state=RANDOM_STATE, verbose=False, thread_count=-1)
        final_model.fit(X_final, y_full)

    artifact = {
        'model': final_model,

```

```

        'model_type': WINNER,
        'threshold': float(best_threshold),
        'feature_cols': list(X_final.columns),
        'numeric_and_ohe_cols': numeric_and_ohe_cols,
        'cat_present': cat_present,
        'target_encoding_maps': full_target_enc_maps,
        'impute_stats': impute_stats,
    }
    joblib.dump(artifact, 'finguard_model.pkl')

with open('feature_list.json', 'w') as f:
    json.dump({'feature_cols': list(X_final.columns), 'threshold': float(best_threshold)}, f, indent=2)

print(f"✓ Saved: finguard_model.pkl ({WINNER.upper()}), feature_list.json")

row = clean_application(row)
row = engineer_application_features(row)
row, _ = impute_and_prepare(row, fit_stats=art['impute_stats'])
row = one_hot_low_card(row, exclude_cols=art['cat_present'])

for col in art['numeric_and_ohe_cols']:
    if col not in row.columns:
        row[col] = 0

if art['model_type'] == 'cat':
    for c in art['cat_present']:
        row[c] = row[c].astype(str) if c in row.columns else 'Unknown'
    X_row = row[art['feature_cols']]
else:
    for c in art['cat_present']:
        enc_info = art['target_encoding_maps'][c]
        val = input_dict.get(c, None)
        row[f'{c}_TE'] = enc_info['map'].get(val, enc_info['global_mean'])
    X_row = row[art['feature_cols']]

prob = float(art['model'].predict_proba(X_row)[0, 1])
threshold = art['threshold']
score = 300 + 550 * (1 - prob)
band = 'Low' if prob < 0.2 else ('Medium' if prob <= 0.5 else 'High')

return {
    'probability_default': round(prob, 4),
    'credit_score': int(round(score)),
    'risk_band': band,
    'decision_threshold_used': threshold,
    'flagged_high_risk': prob >= threshold,
}

print("✓ Inference function ready. Usage: result = predict_score({...})")

```

```

def predict_score(input_dict, artifact_path='finguard_model.pkl'):
    art = joblib.load(artifact_path)
    row = pd.DataFrame([input_dict])

    row = clean_application(row)
    row = engineer_application_features(row)
    row, _ = impute_and_prepare(row, fit_stats=art['impute_stats'])
    row = one_hot_low_card(row, exclude_cols=art['cat_present'])

    for col in art['numeric_and_ohe_cols']:
        if col not in row.columns:
            row[col] = 0

    if art['model_type'] == 'cat':
        for c in art['cat_present']:
            row[c] = row[c].astype(str) if c in row.columns else 'Unknown'
        X_row = row[art['feature_cols']]
    else:
        for c in art['cat_present']:
            enc_info = art['target_encoding_maps'][c]
            val = input_dict.get(c, None)
            row[f'{c}_TE'] = enc_info['map'].get(val, enc_info['global_mean'])
        X_row = row[art['feature_cols']]

    prob = float(art['model'].predict_proba(X_row)[0, 1])

```

```
threshold = art['threshold']
score = 300 + 550 * (1 - prob)
band = 'Low' if prob < 0.2 else ('Medium' if prob <= 0.5 else 'High')

return {
    'probability_default': round(prob, 4),
    'credit_score': int(round(score)),
    'risk_band': band,
    'decision_threshold': threshold,
    'flagged_high_risk': bool(prob >= threshold),
}

# Test inference
test_sample = {
    'AMT_INCOME_TOTAL': 202500, 'AMT_CREDIT': 406597.5, 'AMT_ANNUITY': 24700.5,
    'AMT_GOODS_PRICE': 351000, 'DAYS_BIRTH': -9461, 'DAYS_EMPLOYED': -637,
    'CNT_CHILDREN': 0, 'CNT_FAM_MEMBERS': 2, 'CODE_GENDER': 'F',
}
result = predict_score(test_sample)
print(f"\n=== INFERENCE TEST ===")
print(json.dumps(result, indent=2))
print(f"✓ Pipeline complete!")
```

Start coding or [generate](#) with AI.